JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Interface Era

All Superinterfaces:

TemporalAccessor, TemporalAdjuster

All Known Implementing Classes:

HijrahEra, IsoEra, JapaneseEra, MinguoEra, ThaiBuddhistEra

```
public interface Era
extends TemporalAccessor, TemporalAdjuster
```

An era of the time-line.

Most calendar systems have a single epoch dividing the time-line into two eras. However, some calendar systems, have multiple eras, such as one for the reign of each leader. In all cases, the era is conceptually the largest division of the time-line. Each chronology defines the Era's that are known Eras and a `Chronology.eras` to get the valid eras.

For example, the Thai Buddhist calendar system divides time into two eras, before and after a single date. By contrast, the Japanese calendar system has one era for the reign of each Emperor.

Instances of `Era` may be compared using the == operator.

Implementation Requirements:

This interface must be implemented with care to ensure other classes operate correctly. All implementations must be singletons - final, immutable and thread-safe. It is recommended to use an enum whenever possible.

Since:

1.8

Method Summary

All Methods	Instance Methods	
Modifier and Type	**Method**	**Description**
`default Temporal`	**`adjustInto`**`(Temporal temporal)`	Adjusts the specified temporal object to have the same era as this object.
`default int`	**`get`**`(TemporalField field)`	Gets the value of the specified field from this era as an `int`.

default String	**getDisplayName**(TextStyle style, Locale locale)	Gets the textual representation of this era.
default long	**getLong**(TemporalField field)	Gets the value of the specified field from this era as a long.
int	**getValue**()	Gets the numeric value associated with the era as defined by the chronology.
default boolean	**isSupported**(TemporalField field)	Checks if the specified field is supported.
default <R> R	**query**(TemporalQuery<R> query)	Queries this era using the specified query.
default ValueRange	**range**(TemporalField field)	Gets the range of valid values for the specified field.

Method Details

getValue

```
int getValue()
```

Gets the numeric value associated with the era as defined by the chronology. Each chronology defines the predefined Eras and methods to list the Eras of the chronology.

All fields, including eras, have an associated numeric value. The meaning of the numeric value for era is determined by the chronology according to these principles:

- The era in use at the epoch 1970-01-01 (ISO) has the value 1.
- Later eras have sequentially higher values.
- Earlier eras have sequentially lower values, which may be negative.

Returns:

the numeric era value

isSupported

```
default boolean isSupported(TemporalField field)
```

Checks if the specified field is supported.

This checks if this era can be queried for the specified field. If false, then calling the `range` and `get` methods will throw an exception.

If the field is a `ChronoField` then the query is implemented here. The `ERA` field returns true. All other `ChronoField` instances will return false.

If the field is not a `ChronoField`, then the result of this method is obtained by invoking `TemporalField.isSupportedBy(TemporalAccessor)` passing `this` as the argument. Whether the field is supported is determined by the field.

Specified by:

`isSupported` in interface `TemporalAccessor`

Parameters:

`field` - the field to check, null returns false

Returns:

true if the field is supported on this era, false if not

range

```
default ValueRange range(TemporalField field)
```

Gets the range of valid values for the specified field.

The range object expresses the minimum and maximum valid values for a field. This era is used to enhance the accuracy of the returned range. If it is not possible to return the range, because the field is not supported or for some other reason, an exception is thrown.

If the field is a `ChronoField` then the query is implemented here. The `ERA` field returns the range. All other `ChronoField` instances will throw an `UnsupportedTemporalTypeException`.

If the field is not a `ChronoField`, then the result of this method is obtained by invoking `TemporalField.rangeRefinedBy(TemporalAccessor)` passing `this` as the argument. Whether the range can be obtained is determined by the field.

The default implementation must return a range for `ERA` from zero to one, suitable for two era calendar systems such as ISO.

Specified by:

`range` in interface `TemporalAccessor`

Parameters:

`field` - the field to query the range for, not null

Returns:

the range of valid values for the field, not null

Throws:

`DateTimeException` - if the range for the field cannot be obtained

`UnsupportedTemporalTypeException` - if the unit is not supported

get

```
default int get(TemporalField field)
```

Gets the value of the specified field from this era as an `int`.

This queries this era for the value of the specified field. The returned value will always be within the valid range of values for the field. If it is not possible to return the value, because the field is not supported or for some other reason, an exception is thrown.

If the field is a `ChronoField` then the query is implemented here. The `ERA` field returns the value of the era. All other `ChronoField` instances will throw an `UnsupportedTemporalTypeException`.

If the field is not a `ChronoField`, then the result of this method is obtained by invoking `TemporalField.getFrom(TemporalAccessor)` passing `this` as the argument. Whether the value can be obtained, and what the value represents, is determined by the field.

Specified by:

`get` in interface `TemporalAccessor`

Parameters:

`field` - the field to get, not null

Returns:

the value for the field

Throws:

`DateTimeException` - if a value for the field cannot be obtained or the value is outside the range of valid values for the field

`UnsupportedTemporalTypeException` - if the field is not supported or the range of values exceeds an `int`

`ArithmeticException` - if numeric overflow occurs

getLong

```
default long getLong(TemporalField field)
```

Gets the value of the specified field from this era as a `long`.

This queries this era for the value of the specified field. If it is not possible to return the value, because the field is not supported or for some other reason, an exception is thrown.

If the field is a `ChronoField` then the query is implemented here. The `ERA` field returns the value of the era. All other `ChronoField` instances will throw an `UnsupportedTemporalTypeException`.

If the field is not a `ChronoField`, then the result of this method is obtained by invoking `TemporalField.getFrom(TemporalAccessor)` passing `this` as the argument. Whether the value can be obtained, and what the value represents, is determined by the field.

Specified by:

`getLong` in interface `TemporalAccessor`

Parameters:

`field` - the field to get, not null

Returns:

the value for the field

Throws:

`DateTimeException` - if a value for the field cannot be obtained

`UnsupportedTemporalTypeException` - if the field is not supported

`ArithmeticException` - if numeric overflow occurs

query

```
default <R> R query(TemporalQuery<R> query)
```

Queries this era using the specified query.

This queries this era using the specified query strategy object. The `TemporalQuery` object defines the logic to be used to obtain the result. Read the documentation of the query to understand what the result of this method will be.

The result of this method is obtained by invoking the `TemporalQuery.queryFrom(TemporalAccessor)` method on the specified query passing `this` as the argument.

Specified by:

`query` in interface `TemporalAccessor`

Type Parameters:

`R` - the type of the result

Parameters:

query - the query to invoke, not null

Returns:

the query result, null may be returned (defined by the query)

Throws:

DateTimeException - if unable to query (defined by the query)

ArithmeticException - if numeric overflow occurs (defined by the query)

adjustInto

```
default Temporal adjustInto(Temporal temporal)
```

Adjusts the specified temporal object to have the same era as this object.

This returns a temporal object of the same observable type as the input with the era changed to be the same as this.

The adjustment is equivalent to using Temporal.with(TemporalField, long) passing ChronoField.ERA as the field.

In most cases, it is clearer to reverse the calling pattern by using Temporal.with(TemporalAdjuster):

```
// these two lines are equivalent, but the second approach is recommended
temporal = thisEra.adjustInto(temporal);
temporal = temporal.with(thisEra);
```

This instance is immutable and unaffected by this method call.

Specified by:

adjustInto in interface TemporalAdjuster

Parameters:

temporal - the target object to be adjusted, not null

Returns:

the adjusted object, not null

Throws:

DateTimeException - if unable to make the adjustment

ArithmeticException - if numeric overflow occurs

getDisplayName

```
default String getDisplayName(TextStyle style,
                              Locale locale)
```

Gets the textual representation of this era.

This returns the textual name used to identify the era, suitable for presentation to the user. The parameters control the style of the returned text and the locale.

If no textual mapping is found then the `numeric value` is returned.

API Note:

This default implementation is suitable for most implementations.

Parameters:

`style` - the style of the text required, not null

`locale` - the locale to use, not null

Returns:

the text value of the era, not null

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.